

Magic 600 Cell

1.0 Architecture and Migration Contract

Orbit First Block Building Solving

One authoritative state. A new interactive frontend.

Source revision	2026-09-18 / 1.0-A3
Source status	Proposed architecture for review
Reading edition	18 September 2026
Scope	Integrated contract, release priority and review amendments

This edition integrates the reviewed architecture and the latest release priority. Proposed implementation choices remain proposals; no application or GPU validation is claimed.

Architecture at a glance

Purpose. Make a human-led solution more achievable by reducing buffer preparation, protection checks and repeated verification. The user chooses the solving actions; the application exposes exact state, evaluates effects and preserves recoverability.

One authority, a new frontend

Keep the Python engine, immutable mathematical model and Session journal as the only authority. A Rust shell with egui and dedicated wgpu rendering presents revision-bound snapshots and sends validated commands. Independent windows share the same puzzle state and may retain independent cameras.

Prove one complete solving operation first

Select a real Local object; choose an explicit reference and macro; inspect the actual emitted cost and full protection; commit explicitly; update every view consistently; then undo or recover. This vertical slice is the first useful deliverable. A drawing demonstration alone does not establish it.

What this integrated edition improves

- Durable command receipts resolve uncertain delivery; equal state hashes alone cannot identify a transaction.
- Cancellation distinguishes a request from an actual pre-commit cancellation. A committed operation stays committed.
- Snapshots, labels and styles are published coherently; picking is bound to the correct view and relevant revisions.
- Recorded provenance is distinct from execution permission; incompatible drafts are retained and rollback restores a compatible engine as well as a shell.
- B4-12 stays not passed and deferred into the 1.0 workstream; original thresholds, correctness, protection and recovery remain intact.

Current release comes first

The user accepts deferred closure of the still-unpassed B4-12 performance item for the current release. R00 preserves all other release requirements and the original thresholds. V11.1 schedules optimization and validation after the planned device change; hardware causality is not established. At most one low-risk closeout candidate has a total 90-minute cap; this document does not restart that allowance. The 1.0 migration is not required for today's release.

How to read the contract

[C] is a confirmed constraint; [P] a proposed design; [D] a pending user decision. [F] preserves observations from the original A1 investigation, not a fresh audit. The supplied original is retained; amendments are mapped in Appendix C. Document compilation and visual checks are not native application or performance evidence.

Contents

Architecture at a glance	i
R00. Current release priority and deferred B4-12	1
V00. Authority and reading contract	2
V01. Investigation basis and evidence boundary	2
V02. Process and ownership boundaries	3
V03. Transport and session protocol	4
V04. Certificate and permission lifecycle — N1	6
V05. Compile cost, limits and caches — N2, N3, N4	8
V06. Error taxonomy — N5	11
V07. Naming round trip and discoverable entry — N7	12
V08. Selection roles, cameras and async picking	13
V09. Versioning and compatibility — N6	14
V10. Old/new equivalence — same full-label sample	15
V11. GUI route and switching conditions	16
V12. Migration slices	19
V13. Decisions required from the user	19
V14. Historical source observations and evidence boundary	20
Appendix A. N1-N7 tracking	20
Appendix B. Related documents	22
Appendix C. Integration record	23

Revision: 2026-09-18 / 1.0-A3 (integrated architecture with deferred B4-12) Method: **Orbit First Block Building Solving** (unchanged) Status: **Proposed architecture for review**. No migration is started, no 0.4 product code is modified, no model asset or personal session is touched, and nothing here is a release commitment.

R00. Current release priority and deferred B4-12

R00.1 User-confirmed release decision

[C] **Latest instruction, 18 September 2026:** prioritize completion of the current release. B4-12 performance acceptance remains **not passed and deferred**. The user accepts that open item for this release and places its optimization and verification in the integrated 1.0 Architecture workstream, to resume after the planned device change. Deferral is not a passing result, a lower threshold or proof of a hardware cause. No statement here guarantees that changing equipment will resolve the shortfall.

[C] **All existing performance metrics, timing boundaries, correctness requirements and failed observations remain intact.** This replaces A2's conditional hardware-exception wording. Mathematical correctness, protection, permissions, truthful commit/cancel outcomes, persistence, recovery, independent desktop checks and package/source consistency remain release requirements. Other hard blockers must be reported specifically.

[C] **Scope boundary:** finish the current release without expanding performance attribution, features, unrelated refactoring, visual polishing or protocol redesign. This document does not start the 1.0 migration or a new optimization run. Proposed 1.0 architecture decisions remain proposals and retain their existing approval boundaries.

R00.2 One bounded optional candidate; no restarted allowance

[C] **The current release permits at most one low-risk candidate addressing established duplicate UI work.** Optimization, review and targeted verification together have a 90-minute ceiling. Record its hypothesis, owned files and benefit criterion before editing. Preserve protection, permission, commit and durability semantics. If there is no clear local benefit, a correctness regression, or no completed validation within the limit, withdraw only that candidate and retain the previously verified version. Preserve other existing modifications. This revision does not restart the clock or authorize a second candidate.

[C] **Use small, fixed samples and retain every attempt.** Stop an obviously over-budget candidate's long test; do not repeat runs to select a favorable result. Do not start the three 100-sample campaigns during this deferred close-out. Evidence remains reusable only where the relevant source, dependencies and environment are unchanged; changes require affected checks and necessary regression only.

R00.3 Remaining release obligations

[C] **Performance deferral does not waive any other release requirement.** Complete remaining correctness and independent native desktop acceptance, real packaged operations and recovery, dependency checks and packaging. The final artifact must match its frozen, appropriately verified source. Combine equivalent checks on the same actual packaged workflow where their evidence satisfies both requirements; do not substitute internal fixtures for independent interaction.

[C] **Recording and publication retain their agreed requirements.** The user performs the real key combinations in a concentrated recording session; the assistant organizes the material and edits it. Subtitles remain English only. The current release must not be delayed by starting the proposed 1.0 performance workstream.

R00.4 Public wording and private handoff

[C] **The current application's public release notes use only this performance statement: "Performance optimization is scheduled for the next iteration."** Do not expand those release notes into the current diagnostic investigation, promise target compliance or assert hardware causality. The architecture below is a future engineering plan, not an application release note or a performance certificate.

[C] The internal handoff must preserve the actual failed measurements, original targets, timing boundaries, build and environment identity, all failed attempts, evidence paths, reproduction steps, remaining acceptance items and next-device validation plan. Do not upload raw private logs, tokens, personal sessions, local usernames or absolute machine paths. B4-12 remains separately tracked as not passed/deferred until V11.1's closure evidence exists.

V00. Authority and reading contract

This is the **single authoritative 1.0 architecture entry**. Before it existed, 1.0 material was split across outputs/magic600-v1-gui-study/README.md (candidate study), LEGION_SPIKE.md (spike plan) and the public docs/V1_0_OUTLOOK.md. Those remain valid in their own scope; this file now owns the detailed contracts, and the public outlook carries only a summary pointer. Do not copy full contracts into a second file.

0.4 remains governed by 03_ARCHITECTURE.md, 07_NAMING_AND_RECOMMENDATION.md and 08_INTEGRATION_AND_ENDGAME.md. Where 1.0 says nothing, 0.4 contracts still apply. R00 records the latest user-confirmed B4-12 deferral and release priority; V11.1 owns its follow-up workstream. No section relaxes a mathematical or protection rule.

Every statement below is tagged:

Tag	Meaning
[C]	Confirmed constraint — already decided by the user or by an existing authoritative document
[F]	Source observation at the recorded A1 revision; not a fresh implementation or test claim
[P]	Proposed this round — reviewable engineering decision, not yet approved
[D]	Needs a user decision — do not implement past this point

[C] Reading code is not a passing test. Every **[F]** below was established by reading source at the revision recorded in V01. A2 integrates review proposals without re-auditing those sources. None of it asserts that any test currently passes, that any build is current, or that any candidate is integrated.

V01. Investigation basis and evidence boundary

[F] Source of record resolved through team-project.json ({"project_name":"Magic 600 Cell","source_root":"."}) to the active local source checkout, experiment area work/experiments/magic600-04. Files read this round, with device mtime:

File	mtime (ms)	Used for
reference_variants.py	1789568955221	N1, N2, N3, N4
mathematical_names.py	1789560633580	N7
macro_library.py	1789570591862	N1, N6
adapter.py	1789586142705	N1, N5, selection roles, transport
session.py	1789576381074	transactions, preview/commit
core.py	1788912607795	cancel protocol
server.py, native_bridge.py	1788912608707 / 1788912608243	transport, MPUlt handshake
tests/test_reference_variants.py	1789569065049	N2, N3 coverage
tests/test_mathematical_names.py	1789554970400	N6, N7 coverage
tests/test_mathematical_name_inputs.py	1789555373895	N7 coverage

[F] The public GitHub repository is a documentation archive for 0.4, not a source mirror. docs/progress/0.4/README.md states application code, binaries and raw evidence are excluded. The repository root is the 0.3-era

published tree. `reference_variants.py`, `mathematical_names.py`, `adapter.py` and the 0.4 tests exist **only in the local checkout**. Any 1.0 work that assumes the public repo is the code baseline is wrong.

[F] There is uncommitted local work. `session.py` is 31,217 bytes locally against 28,722 in the published 0.3 tree; `preparation.py` and `grips.py` changes exist locally with no published counterpart. HEAD is not a complete workspace identity — this matches the GUI study's own note.

[F] The `reference_variants.py` reviewed in the previous round is byte-identical to current source (245 lines; the apparent size difference was CRLF). The earlier review conclusions stand against current source.

V02. Process and ownership boundaries

V02.1 Processes

[C] One authoritative puzzle state, one transaction/journal owner. The renderer presents state; it never decides legality and never commits a mathematical change.

[P] 1.0 runs **two processes**, not one:

Process	Owns	Never owns
Engine (Python, existing)	Model (immutable assets), PuzzleState, Session (SQLite journal, preview/commit, undo/redo, checkpoints), Workbench workspace w, macro library, naming, reference variants, endgame library, protection evaluation, scoring	Windows, cameras, hover, GPU resources, layout
Shell (Rust, new)	Windows and viewports, docking, input routing and keymap state, cameras per view, hover, GPU device/queue/pipelines, meshes, picking buffers, draft <i>text</i> being typed	Puzzle state, legality, protection verdicts, canonical identity assignment, execution permission

[P] The shell keeps no second copy of puzzle state that it can mutate. It holds an immutable snapshot plus a revision token. Anything that would change state is a command to the engine.

[F] This split already exists in 0.4 and is load-bearing: `server.py` owns Session; `adapter.py` Workbench owns the workspace; the WinForms/MPUI native host holds only windows, input and drawing. 1.0 replaces the third element, not the first two — consistent with the confirmed constraint that a GUI change is not a reason to rewrite the backend.

V02.2 Renderer submodule boundaries

[C] egui for widgets, text and 2-D interactive graphics. eframe/native viewport for the application and real system windows. egui_dock for **in-workbench docking only** — a floating egui window is not a system window and must never be presented as one. A custom wgpu module for real puzzle/Local/Global projection, transparency, animation, picking and compositing.

[C] 259,800 stickers are never GUI controls. Adopting wgpu does not by itself constitute a dedicated renderer.

[P] Inside the shell:

Module	Responsibility
shell-app	eframe application, viewport lifecycle, layout persistence, DPI/monitor handling
shell-dock	egui_dock trees; explicit dock $\rightleftharpoons$ detach transitions that create/destroy a real viewport
shell-views	per-view state: camera, filter, detail level, follow/pin mode

Module	Responsibility
render-core	wgpu device/queue, immutable geometry upload, bind groups, shared resources
render-puzzle	projection, material, transparency, animation, colour pass, integer-ID pass
render-pick	pick request/response with snapshot + camera version stamping
engine-client	transport, subscription, command submission, cancel, reconnect, revision binding
domain-view	typed mirror of engine DTOs; no derived mathematics

[P] domain-view computes nothing mathematical. It may not recompute an orbit, a protection verdict, a net action, a name, or a score. If the shell needs a derived value, it asks the engine. This is what keeps “the renderer only presents state” enforceable rather than aspirational.

V03. Transport and session protocol

V03.1 What exists now

[F] Current transport is **local HTTP on 127.0.0.1** (server.py): bearer token in a query/header, Host validation, Origin rejection, size limits, a single ThreadPoolExecutor(max_workers=1), a job dictionary, and GET /api/job/<key> returning { 'done': True, 'result': ... } or { 'done': True, 'error': str(exc) }.

[F] The experiment boundary (adapter.py) adds POST /api/experiment/command, .../native-command, .../native-input, .../window-layout, .../session-log, and GET /api/experiment/snapshot, .../native-snapshot?since=, /api/structure. All solving commands serialize under a single context['lock'].

[F] The MPULt-specific coupling is confined to the native profile. native_bridge.verify_native requires a handshake document of format='MPULt-native600-v1' exported from the actual MPULt process, and produces C600-NATIVE-PROFILE-v1 containing native_to_lab, native_face_to_lab, translations, token_words, native_executable_sha256 and profile_sha256. native-input then requires a matching profile_sha256 and state_hash, resolves native_sticker → native_to_lab[slot] → model.sp[lab] → position, and maps input tokens through token_words.

[P] This is the entire MPULt dependency in the protocol, and it is replaceable without touching the mathematics. The canonical layer beneath it (slot, position, identity, orbit) is already MPULt-free. 1.0 does not need a native profile at all: the shell addresses objects by canonical slot/position/identity directly, and twists by explicit generator words — the same representation token_words expands into. native_to_lab remains only for the 0.4 comparison harness (V10), never as a 1.0 runtime path.

V03.2 Proposed 1.0 transport

[P] Keep local HTTP/1.1 on 127.0.0.1 for commands and queries; add one Server-Sent Events stream for notifications; use the existing binary endpoints for bulk arrays. Reasons, stated concretely rather than as preference:

1. **It already works and is already hardened.** Host/Origin/token/size checks, job submission, cancel and the busy-409 are implemented and exercised. A new IPC mechanism would re-litigate all of it for no mathematical gain.
2. **Bulk data is already binary.** /api/labels (<u4>), /api/styles, /api/cap return raw application/octet-stream. 259,800 u32 labels is ~1.04 MB — a loopback HTTP body, not a serialization problem. JSON is used only for structured, small payloads.
3. **Language-neutral.** A Rust shell needs an HTTP client and an SSE reader, both small. Embedding CPython in the Rust process would put the authoritative state inside the shell's crash domain, which contradicts V02.1.
4. **Debuggable.** The protocol stays inspectable with ordinary tools during migration, which matters while two frontends must be compared.

[D] Decision required: transport choice is a durable commitment. Local HTTP + SSE is the proposal. The alternative worth naming is a length-prefixed pipe protocol over the existing EngineProcess parent pipe (lower latency, no port, but a new framing layer and new auth story). This is listed in V13 as decision **D-1**.

[P] Endpoint shape (all under an unambiguous 1.0 prefix so 0.4 routes keep working during comparison):

Kind	Route	Contract
Snapshot	GET /api/v1/snapshot	Complete typed state + revision. Idempotent.
Delta	GET /api/v1/snapshot?since=<revision>	Fields changed since revision; full snapshot if the token is unknown or too old. Never a partial that the client must merge blindly.
Bulk	GET /api/v1/labels, .../styles, .../geometry	Geometry binds to model_id + content hash; mutable arrays also carry the exact committed snapshot revision.
Query	GET /api/v1/name/resolve?text=, .../effect?macro=	Read-only, cancellable, no state change.
Command	POST /api/v1/command	Accepts a scoped command_id and request digest; returns {job} or the existing command receipt. Same ID with different content is rejected.
Job	GET /api/v1/job/<id>	{state, result?, error?} — typed (V06).
Cancel	POST /api/v1/job/<id>/cancel	Requests cancellation; acceptance is not completion. Report cancelled only before durable commit; after commit return the committed result.
Events	GET /api/v1/events (SSE)	revision-changed, job-state, engine-shutdown. Advisory only — never carries authoritative state.

[P] SSE is advisory. A client that misses an event must still converge by polling ?since=. No correctness may depend on event delivery.

V03.3 Revision binding

[C] Old previews, async computations and delayed picking results must never be applied to a mismatched new state.

[F] The source engine already carries these base identifiers: Session.rev, Session.head, st.hash (full-state hash), model_id, and per-object revisions (cell_status computes revision over model_id + interaction revision; filter_preview returns state_hash, revision and context_hash).

[P] Define one composite StateRevision; keep schema compatibility separate from workspace mutation:

```
StateRevision = { model_id, session_id, engine_epoch, head, rev, state_hash,
workspace_revision, review_context_id }
WorkspaceSchema = { schema_version }
```

Rules:

- Every result identifies the state against which it was computed; state-changing permissions require an exact match to their engine-validated dependencies.
- Publish snapshot metadata and mutable label/style arrays as one revision-bound bundle. A client installs the complete matching bundle atomically; missing, mismatched or out-of-order data triggers a validated refresh, never a mixed frame.
- A schema version describes the format; workspace_revision tracks edits. engine_epoch distinguishes process lifetimes. Restart invalidates transient permissions but does not erase durable command identity.
- Read-only caches and picking use declared dependency revisions. Unrelated layout edits must not invalidate an otherwise valid geometric query. Stale hover/picks are discarded quietly; explicitly requested stale work receives a visible explanation.

- Geometry binds only to `model_id` + content hash because it is immutable. Visibility, interaction and annotation revisions remain distinct where they affect a result.

V03.4 Process lifecycle

[F] EngineProcess already owns local engine startup, authenticated health checks, graceful shutdown and parent-pipe recovery, and AGENTS.md mandates its use. `POST /api/shutdown` requires a matching `launch_id`.

[P] The shell owns the engine as a child process: spawn with a fresh token and `launch_id` on a private port; readiness by polling `/api/health`; graceful stop via `/api/shutdown` with the `launch_id`; if the shell dies, the engine exits on parent-pipe EOF. **[P]** The shell never starts an engine against a session directory already held — `session_lock.py` remains the arbiter, and a lock conflict is a first-class typed error, not a crash.

V03.5 Reconnect and unknown-outcome recovery

[C] Saving a workspace never saves a reusable execution permission. A committed state with no receipt delivered must not be silently re-committed.

[F] Source observation: `preview()` returns pre/post full-state hashes and a single-use token; `commit()` checks head, rev and `pre_state`. The pending permission is memory-only. These are useful consistency guards, but state equality does not identify a particular transaction: a legal no-op can have equal hashes, and undo or later operations can revisit an earlier state.

[P] Recover from durable command identity, using hashes as consistency checks:

1. Before submission the shell retains a `command_id`, request digest and session/model scope. Retrying delivery uses the same ID. The engine rejects reuse with different content.
2. The engine durably associates the committed command ID, request digest, journal event and pre/post state hashes in the same recoverable transaction boundary. A transient job table alone is insufficient.
3. After reconnect the shell obtains the current revision and asks for that command's authoritative receipt. A committed receipt means **already committed**, even if later undo/redo changed the current state; never replay it automatically.
4. A durable rejection or cancellation before commit is **not committed**. Offer a fresh preview where appropriate. A still-running or live-preview response is process-epoch-bound and never resurrects an expired token.
5. No receipt, broken journal continuity or an incomplete recovery record means **unknown outcome**, not proof of non-execution. Enter read-only recovery and reconcile the journal. Retry execution only when the engine can authoritatively guarantee it did not commit.
6. A lost success response, a no-op, restart and undo back to an earlier hash must all preserve this rule: one command ID produces at most one committed operation.

[P] Cancellation is phase-aware. Acknowledging a cancellation request does not guarantee cancellation. Before commit, return cancelled with the state unchanged; after durable commit, return the committed receipt. An unresolved crash boundary remains `unknown_outcome`. Never report committed work as cancelled or rolled back.

[P] Do not persist reusable preview permission. Persisting command outcomes is different from persisting an execution token. Receipt lookup and restart recovery are new protocol proposals, not claims that the current engine already implements them.

V04. Certificate and permission lifecycle — N1

V04.1 What the code actually does

[F] Reference variants are **independent saved library entries**, not temporary instances. `adapter.create_macro_variant` with `kind='geometry'`:

- calls `ReferenceVariants.compile(source, source_frame, destination_frame);`

- takes `reference_proof['recipe']` as the new recipe;
- sets `derived_from = {id, version, kind:'geometry', reference, proof}`;
- mints a fresh identity user-`<12 hex>` at `version=1`;
- round-trips the record through `export_selected` (which re-validates through `_record` $\rightarrow$ `_derived`);
- inserts into `w['personal_macros']` and `self.library`, then `save()`, **with full rollback** (`self.w, self.library = previous_work, previous_library`) if saving raises.

[F] `macro_library._derived` requires exactly `{id, version, kind, reference, proof}` for `kind='geometry'`, verifies `reference['model'] == model.model_id` and that both frames are proper ordered frames of their cell, and then stores proof as an **explicitly untrusted record**: the source comment is “*Geometric proof metadata must be a record; it is not trusted on import*”, and for the endgame family “*Provenance is descriptive. Import never grants a frame or action certificate.*”

[F] `compile` emits exactly three certificate tiers, increasing in scope:

Call	Returns	Self-declared scope
<code>inspect</code>	<code>_summary</code> only	Exact fixed-position incidence correspondence; not a legal puzzle move
<code>map_positions</code>	<code>_summary</code> + objects (≤ 256 positions, slot-by-slot)	same, plus explicit object correspondence
<code>compile</code>	<code>_summary</code> + recipe + primitive correspondence + proof	All labelled slots, including every collateral orbit

[F] `_summary` carries `model`, `reference_version='exact-incidence-reference-v1'`, `manifest_sha256`, `source_frame`, `destination_frame`, `slot_map_sha256`, `labels`, `positions`, and four booleans (`cell_bijection`, `slot_bijection`, `piece_consistency`, `orbit_preservation`). `proof` carries `complete_action_equal`, `labels`, `support_labels`, `affected_orbits`, `source_recipe_sha256`, `emitted_recipe_sha256`, `scope`.

[F] `compile` already computes cross-orbit collateral: `affected_orbits=sorted(set(map(int, model.so[actual_source])))`.

[F] `compile` sets `prefix_semantics='Only net effects correspond. Review the actual emitted word for strict prefix protection; original approval and cost are not inherited.'`

[F] `comparison = None` if `kind == 'geometry'` else `self.compare_macros(source, record)` — geometry variants get no `compare_macros` result, because `compile` already proved net-action correspondence by a stronger route.

[F] Execution permission is a separate, non-inheritable object: the preview token bound to `(head, rev, pre_state)`, single-slot, memory-only, re-validated at `commit`, which additionally re-checks `any(r['orbit'] in self.prefs['protected'] for r in p['public']['support'])`.

V04.2 The four states are already structurally distinct — with two gaps

[P] Name them explicitly and give each an independent lifetime:

State	Produced by	Invalidated by	Persisted?
A — reference map verified	<code>_mapping</code> + <code>_summary</code>	<code>model_id</code> , <code>manifest_sha256</code> , <code>reference_version</code> , either frame	Yes, in <code>derived_from.reference</code>
B — emitted net action verified	<code>compile</code> proof	A, plus <code>source_recipe_sha256</code> / <code>emitted_recipe_sha256</code>	Yes, in <code>derived_from.proof</code> — untrusted on load
C — current target and protection check passed	<code>review</code> /effect against live <code>w</code> + <code>prefs['protected']</code>	any state change, orbit switch, role change, protection change, goal change	No — must never be persisted

State	Produced by	Invalidated by	Persisted?
D — valid preview/commit permission	Session.preview token	head, rev, pre_state change; commit; cancel; engine restart	No — memory only

[P] Gap 1: emitted cost is not persisted. compile returns source_primitives and emitted_primitives, but derived_from stores neither — only reference and proof, and proof has no cost field. The brief requires the actual emitted cost to be bound. **Proposal:** add emitted_primitives and source_primitives to the proof record and to _derived's accepted key set. This is an additive schema change (V09) and needs the compatibility rule below.

[P] Gap 2: A and B are checked at save time but _derived does not re-verify them at load time, by design (proof is untrusted). That is correct, but it means a loaded geometry variant is a macro **with provenance**, not a macro **with a live certificate**. The UI must therefore show A/B for a loaded variant as *recorded, not re-verified*, until the engine recomputes. **Proposal:** the engine exposes POST /api/v1/macro/reverify which recomputes A and B for a stored variant against the current model and returns a fresh certificate; the record's displayed state is recorded until that returns verified.

V04.3 Availability rules

[C] Buttons, keyboard and API obey one availability rule. A warning sentence is never the mechanism that prevents a mistaken execution.

[P] One engine-computed availability object, consumed identically by every input path:

```
Availability = {
  save: bool, # needs: valid record, library capacity
  add_to_draft: bool, # recorded provenance may enter a draft; orbit must match
  # execution still requires current verified A/B/C and live D
  review: bool, # needs: draft non-empty, model match
  preview: bool, # needs: C computed and not conflicting
  commit: bool, # needs: D live and C still valid at this revision
  reasons: [ {gate, code, detail} ] # typed, per V06
}
```

[P] The shell renders a control as disabled **iff** the engine says so. The shell never computes availability locally, and never enables a control optimistically. Keyboard shortcuts and the onscreen keyboard route through the same command layer and therefore inherit the same gate — this matches the existing 0.4 requirement that every new feature joins the unified command/visible set.

[P] Original approval and cost never transfer. When a variant is created, the new record starts with no protection approval, no Strict audit, and its own emitted_primitives. create_macro_variant already mints a new identity at version=1 and drops the source's metadata except through derived_from; the UI must not display the source's use tags, notes or pins on the variant. **[F]** 0.4 already established this rule for the R⁻¹ / Macro / R variant ("new variants must not wrongly inherit the source's forward use labels, notes or pins"); 1.0 extends it to geometry variants.

V05. Compile cost, limits and caches — N2, N3, N4

V05.1 Limits are two different things

[F] MAX_PRIMITIVES = 3_000_000 is a total-operation ceiling; WORD_LIMIT = 100_000 is a representation chunk size. The check is if len(emitted) + len(word) > MAX_PRIMITIVES: raise, then recipe = [dict(kind='word', moves=emitted[i:i+WORD_LIMIT]) ...]. Chunking runs after the ceiling test and cannot raise the ceiling. **[F] This is**

already asserted: `test_emitted_limit_and_chunking_use_actual_cost_without_truncation` patches `WORD_LIMIT` to 3 and gets chunk lengths [3,3,2] with `emitted_primitives == 8` — total unchanged.

[P] Never present chunking as a way past the ceiling in UI text, tooltips or errors.

[F] **Minor defect:** the ceiling message hardcodes the literal 3,000,000 rather than interpolating `MAX_PRIMITIVES`, so a patched or future limit reports the wrong number. Visible in the test above, which asserts on '3,000,000' while the limit is 7. [P] Interpolate the constant. Low priority, but it is exactly the kind of message a user would act on.

V05.2 Cost knowledge by stage

[P] Three stages with honest labels — no pseudo-precise numbers before they are earned:

Stage	Available	Label
Frames not fully chosen	source primitive count only	Exact for source; emitted Unknown
Frames chosen, not compiled	exact emitted cost, via the pre-pass below	Exact , marked <i>predicted, not yet built</i>
Compiled	<code>source_primitives</code> , <code>emitted_primitives</code>	Exact , built

[P] **An exact pre-pass is available and cheap, and it does not allocate the emitted sequence.** Cost is a sum of per-primitive word lengths:

$$\text{emitted_cost} = \sum \text{over expanded source moves } m \text{ of } \text{len}(\text{word}(m))$$

`word(m)` depends only on the mapping and `m`. Distinct `m` is bounded by ± 1200 generators, so the pre-pass computes at most 2400 words once per mapping, then accumulates lengths as integers. Memory is $O(\text{distinct moves})$, not $O(\text{emitted})$. `model.expand(normalized)` is already streamed in the current loop, and `model.check_cancel()` is already called every 128 iterations, so the pre-pass is cancellable on the existing protocol. [P] Expose it as `GET /api/v1/reference/cost?macro=&source_frame=&destination_frame=` returning `{source_primitives, emitted_primitives, exceeds_limit, distinct_moves}`.

[P] **Over-limit handling preserves everything and changes no mathematics.** On `exceeds_limit`, the shell keeps the source macro, both frame choices and the draft, shows the exact overrun, and offers only: choose different frames, choose a different source macro, or cancel. It does **not** offer automatic splitting.

[F] **“split the source operation” has no tool support.** A search across `adapter.py`, `macro_use.py`, `macro_library.py` and `work_sheets.py` found no split/segment capability. [P] Therefore the phrase must not appear in user-facing text as a recovery path.

[P] **And splitting is not a neutral refactor.** One atomic operation split into several commits changes: the protection check (Strict is a path predicate over the *actual executed* prefixes — `08` and the theory background both require this, and a split creates new committed intermediate states that were never prefix-checked as one operation), the transaction boundary, the journal shape, undo granularity, and the intermediate states another person or a later session can observe. [P] Splitting is therefore a **user-initiated mathematical decision**, not a compiler fallback, and is out of 1.0 scope unless explicitly requested — listed as **D-4**.

V05.3 Cancellation and cache safety — N3

[F] **Cancellation is `InterruptedError`, not `ValueError`.** `core.Model.check_cancel` raises `InterruptedError('Analysis cancelled; puzzle state unchanged')`. `cancel_event` is owned by the request handler: `adapter` sets `context['model'].cancel_event = cancel` inside `work()` under `context['lock']` and clears it in `finally`. Lifetime is exactly one job.

[F] **There is no concurrency inside `ReferenceVariants`.** One `Workbench` service instance (`adapter.py:1793`), one `ReferenceVariants` constructed once in `Workbench.__init__` (`adapter.py:101`), one `context['lock']`, and `pool = ThreadPoolExecutor(max_workers=1)` in `server.py`. Solving commands are fully serialized; window-layout is the one route that uses `acquire(blocking=False)` and yields immediately rather than queueing behind a solving job.

[F] **Each cache is published only when complete:**

Cache	Assigned	Complete at assignment?	Mutable contents?
<code>_caps[cap]</code>	end of <code>_cap_words</code> , after the <code>len(words)!=12</code> check	Yes	Yes — queue arrays have no <code>setflags(write=False)</code>
<code>_base</code>	end of <code>_base_regions</code> , after every rotation validated	Yes	No — each values array is <code>setflags(write=False)</code>
<code>_maps[key]</code>	only via <code>_publish</code> , called after a fully validated mapping	Yes	No — cells, slots, positions are <code>setflags(write=False)</code>

[F] A partially built mapping can never be observed, and this is already tested. `test_cancel_does_not_publish_a_partial_mapping_or_variant` asserts `_maps` is empty after a cancelled compile; `test_mid_compile_cancel_retains_only_an_existing_incidence_map` asserts the pre-existing mapping is the *same object* and still has `primitive_words == {}`; `test_complete_oracle_rejects_a_wrong_emitted_word_without_publication` asserts nothing is published when the emitted word disagrees.

[F] The temporary primitive map does not share a mutable reference with a published mapping. `compile` does `cache = dict(mapping['primitive_words'])` (a copy), fills `cache`, and only on success does `completed = dict(mapping, primitive_words=cache)`; `self._publish(completed)` — a new dict. `_primitive` stores `cache[move] = list(word)` (a copy of the cached word), and `compile` returns `deepcopy(result)`.

[P] Contract — cancel point → retained objects → next call:

Cancel point	Retained	Next call behaviour
During <code>_cap_words</code>	nothing for that cap	recomputes from scratch
During <code>_base_regions</code>	<code>_caps</code> (complete)	recomputes <code>_base</code>
During <code>_mapping</code>	<code>_caps</code> , <code>_base</code>	recomputes that mapping
During compile primitive loop	<code>_caps</code> , <code>_base</code> , and any previously published mapping with its original <code>primitive_words</code>	recomputes emitted words; no partial reuse
After proof, before <code>_publish</code>	as above	recomputes

[P] Two hardening items, neither a behaviour change: mark the `_cap_words` queue arrays read-only for consistency with the other two caches; and key `_caps/_base` by `model_id` so a model swap cannot silently reuse them.

[P] Verification plan: extend `test_reference_variants.py` with a cancel during `_base_regions`, a post-proof/pre-publish cancel, and an assertion that cached arrays reject writes. **No implementation this round.**

V05.4 The four-entry LRU — N4

[F] `_maps` is capped at 4 by `_publish` (while `len(self._maps) > 4`: `popitem(last=False)`), with `move_to_end` on hit. Its scope is one `ReferenceVariants` instance = one `Workbench` = one engine process. Windows do not create service instances; the 0.4 native frontend is multiple windows against one HTTP server. **Four slots therefore has nothing to do with a window count.**

[P] The cache is a pure performance policy with no correctness role. Every entry is a deterministic function of `(model, source_frame, destination_frame)`; eviction can only cost recomputation. Eviction must never invalidate a user's selection, a saved variant, or a mathematical object — and it currently cannot, because saved variants carry their own `derived_from` and do not reference cache entries.

[P] Memory per entry, from the code: `cells 600×i4 = 2.4 KB`; `slots 259,800×i4 = 1.04 MB`; `positions 177,120×i4 = 708 KB`; plus `primitive_words` (≤ 2400 words, list-of-int, the variable part). So ≈ 1.75 MB plus words — four

entries ≈ 7 MB before `primitive_words`. That is small, which is precisely why **[P] the capacity must not be raised without a measurement**: the justification would have to come from an observed multi-window hit-rate, not from headroom. **[P] Evidence needed before changing it**: hit/miss counters per mapping key under the V12 slice with Global + Local + abstract + keyboard open, over a real cross-orbit sequence.

[P] 1.0 cache contract: owner = engine, single instance; key = (model_id, reference_version, source_frame, destination_frame); source/destination order is preserved, never sorted; concurrency = engine lock, single writer; capacity = 4 until measured; eviction = never user-visible.

V06. Error taxonomy — N5

[F] The gap is real and it is at the transport boundary, not in the domain. Domain-level rejection reasons are already structured — adapter returns `dict(reasons=..., roles=role_status, direction=direction, frame=frame_status)` and `invariant_status['reason']`. But exceptions are flattened: both `server.py` and `adapter.py` end with `except (ValueError, KeyError, TypeError) as error: return self.js({'error': str(error)}, 400)`, and `GET /api/job/<key>` returns `{'done': True, 'error': str(exc)}`. The only machine-readable discriminators today are HTTP status codes: 400 (all input/domain errors), 403 (auth/origin), 404, 409 ('An operation is busy; input was not queued. '), 413, 415, 500, 503.

[F] Consequence: cancellation is indistinguishable from a protection conflict without matching English text. `InterruptedError` escapes the `ValueError` handler and reaches the job endpoint's `except Exception`, arriving as a string like any other failure.

[F] The 0.4 tests themselves depend on English substrings — `test_wrong_type_model_version_and_cross_orbit_preserve_pending_work` uses `assertRaisesRegex` against 'Piece', 'model', 'version', 'region', 'versioned', 'orbit'. This is direct evidence that the string is currently load-bearing.

[P] Introduce a typed error envelope; keep the message as a human summary only.

```
{ "code": "<stable machine token>",
  "category": "input | unverified_reference | protection | stale | cancelled |
resource_limit | consistency | unknown_outcome | auth | busy",
  "recoverable": true|false,
  "retry": "never | after_refresh | after_user_change | after_reconnect",
  "context": { ... typed fields: orbit, position, gate, limit, actual ... },
  "summary": "<short user-facing sentence>",
  "diagnostic": "<expandable internal detail>" }
```

[P] The envelope has ten categories. Eight domain categories and their continuations are listed below. `auth` requires an authenticated reconnect; `busy` requires waiting for the active job or an explicit user cancellation, never blind command replay.

Category	Example from current source	Continuation
input	'Choose an explicit cell C1..C600 and ordered four-corner frame'	fix the field; draft preserved
unverified_reference	'The selected ordered references do not determine one proper correspondence'	choose the disambiguating frame; not a mathematical impossibility
protection	'Protected orbit would move...'	change protection explicitly, or choose another macro
stale	'Stale preview. Preview the operation again.', 'Native view is stale; refresh before input'	refresh and re-review; inputs preserved
cancelled	<code>InterruptedError('Analysis cancelled; puzzle state unchanged')</code>	nothing to fix; resume when ready

Category	Example from current source	Continuation
resource_limit	'Compiled reference exceeds the existing 3,000,000-primitive limit'	V05.2 flow
consistency	'Complete emitted macro differs from the explicitly chosen reference transport'	internal failure; stop and report, never retry silently
unknown_outcome	unresolved durable command receipt (V03.5)	read-only recovery; no execution

[C]/[P] Unknown is not a synonym for “an exception happened”. It means a specific conclusion lacks sufficient evidence — an orientation with no certified frame, an unverified Pure claim. It is a **property of a result**, never an error category. Accordingly the shell models three independent things and never merges them: **evidence state** (verified | recorded | unknown | unchecked), **score** (UseScore / CurrentScore), and **permission** (V04.3 Availability). A hard constraint failure must never render as a low score, and unknown must never render as zero.

[P] The frontend never string-matches. code is stable and versioned; summary is free to change and to be translated. **[P] The 0.4 tests’ assertRaisesRegex calls should migrate to asserting code** once the envelope exists — that migration is the acceptance evidence that the string is no longer load-bearing.

[P] Internal diagnostic vs. user-actionable is a boundary, not a formatting choice. diagnostic may carry hashes, slot indices and internal tokens; summary may not require them to be understood. Draft, frame selection and valid context survive every category above except consistency and unknown_outcome, which freeze execution but still preserve the draft.

V07. Naming round trip and discoverable entry — N7

[F] The parse API exists and is exact. mathematical_names.py provides parse_slot, parse_address and parse_identity, all routing through _parse_address(text, kind). copy_text is produced by _copy_address as four | -separated fields:

```
Magic600.<Slot|Position|Piece> | <naming_version> | <model_id> | <cell pole> / <structure class> /
<region word>
```

NAMING_VERSION = 'incidence-address-v1'. Parsing rejects, with distinct messages: non-text; wrong field count; wrong kind (“object types are not interchangeable”); version mismatch; model mismatch; unknown cell pole; unknown structure/region word. **[F]** Display spelling is normalized on input — τ^{-1} maps to the internal τ^{-1} token — so both the new display form and older copied text parse to the same object.

[F] Support matrix, as implemented:

Input form	Slot	Position	Piece	Note
Full copy_text (4 fields)	Yes	Yes	Yes	the only reversible text form
Bare math address (pole / class / word)	No	No	No	_parse_address requires all four fields
Short display name (identity_short)	No	No	No	display only
Legacy integer ID	Yes	Yes	Yes	separate path, still strict
Fuzzy / search	—	—	—	not implemented

[P] Therefore: never claim any displayed string is reversible. Only copy_text and the legacy integer are. Short names are for recognition, not for retrieval, and fuzzy search — if ever added — is a *locator* that must produce a candidate list the user confirms, never a parser.

[F] The failure contract is state-preserving and tested. test_wrong_type_model_version_and_cross_orbit_preserve_pending_work establishes a pending preview and a locked Next, then drives ten rejection cases asserting

both `self.unchanged() == before` and that the caller's input dict was not mutated. `test_piece_copy_keeps_identity_while_position_copy_stays_fixed_after_turn` commits a real turn and shows Piece copy_text still resolves to the same identity while Position copy_text stays fixed. `test_existing_integer_inputs_remain_compatible_and_strict` covers legacy integers and rejects True, -1, `model.np`, 1.0, None.

[F] Aliases are handled: `test_aliases_of_one_hosted_position_do_not_create_distinct_buffers` asserts that different aliases of one hosted position do not produce distinct buffers.

[P] Remaining coverage gaps, stated precisely (these are gaps in *evidence*, not known defects):

1. **Global uniqueness of identity_short is not asserted.** `test_compact_names_include_home_cell_and_region` checks 433 slot short names, two sampled pieces, and 120 vertex pieces — not all 177,120.
2. **Legal-action invariance is sampled:** `test_same_signature_survives_legal_actions` uses 6 of 1200 generators, ~32 points each.
3. **No model_id/manifest assertion in the naming tests.** They construct `Model()` directly, so “which model version did this evidence bind to” is answerable only from the build record (see N6/V09).
4. **UI end-to-end is out of scope by the file's own docstring** — “*these are not native UI or human solving trials.*”

[P] “A parse API exists” and “a user can find and use it” are two acceptance levels. 1.0 must demonstrate the second: every object that displays a name offers `copy`; every field that accepts an object accepts `copy_text` and the legacy integer; a resolved object scrolls the real Local/Global view to it and selects it; a failure names the category (V06) without clearing the draft. **[F] This is not yet satisfied in 0.4** — the independent solver review recorded a raw-ID workaround (S3), and the memory explicitly states “S3 raw 编号绕行不算可发现性通过”; the C3 candidate (mathematical-name orbit-protection entry) is one of three **unmerged** fixes. **[P] The round trip name → canonical object → graphic location → name is a 1.0 acceptance item (V12 slice 1), not an inherited pass.**

V08. Selection roles, cameras and async picking

[F] The workspace already separates the roles; there is no selectedId. `Workbench.new_workspace` defines `current`, `target`, `next` (`{identity, target}`), `inspected`, `inspected_position`, `goal`, `reference` (legal word), `roles` (`[A, B]`), `block` (`{name, members, protected}`), `orbit`, `bank`, `phase`, `draft` (per phase), `filter`, `saved_filters`, `keybinds`, `view` (`{tab, local, global_view, local_center}`), `contexts`, `captures`.

[F] Each action writes a specific, disjoint field:

Action	Writes	Notes
<code>inspect</code>	<code>inspected</code> , clears <code>inspected_position</code>	Piece-typed input
<code>inspect-position</code>	<code>inspected_position</code> , clears <code>inspected</code>	Position-typed; mutually exclusive with above
<code>focus</code>	<code>current</code> , <code>target</code> , <code>inspected</code>	also calls <code>switch_orbit(o, restore=True)</code>
<code>next-pin</code>	<code>next</code>	refuses to overwrite unless <code>replace=True</code>
<code>next-clear</code>	<code>next</code> → None	
<code>goal</code>	<code>goal</code>	
<code>roles</code>	<code>roles</code>	two distinct positions, same orbit
<code>reference</code>	<code>reference</code>	explicit legal word, <code>[]</code> = canonical
<code>block-add/remove/protect</code>	<code>block</code>	
<code>protect</code>	<code>prefs['protected']</code>	Session-level, not workspace

[F] Two couplings worth designing around: setting `Current` can change the working orbit and restore that orbit's draft; and `next-pin` already enforces explicit replacement — the “locked Next” is not silently overwritten.

[P] 1.0 selection model — seven roles, three owners:

Role	Owner	Shared across windows?	Survives window close?
hover	Shell, per view	No	No
inspect (inspected / inspected_position)	Engine	Yes	Yes
current + target	Engine	Yes	Yes
next	Engine	Yes	Yes
goal	Engine	Yes	Yes
reference	Engine	Yes	Yes
grip	Shell (latched input), mirrored to engine on twist	Yes, as display	No (held input cleared)

[P] hover never leaves the shell and never reaches the engine. It is the only role that is per-view, and keeping it local is what prevents hover traffic from touching the authoritative state or the job queue.

[P] Cameras are per view and never shared. Camera changes are display-only and never alter puzzle state — a confirmed constraint. `view.local_center` stays in the engine workspace because it selects *which mathematical neighbourhood* is shown, not a viewing angle. This distinction is the rule: **what to show is engine state; how to look at it is shell state.**

[P] Window close retains engine-owned roles and discards shell-owned ones. Reopening a view restores layout and camera from persisted preferences (`session.prefs[ 'layout' ]`, already used for the workspace) and re-subscribes to current state. Closing the last view of a kind never clears current, next or roles.

[P] Async picking invalidation. Every pick carries `{view_id, view_generation, StateRevision, camera_version, viewport_size, visibility_revision, interaction_revision}`. Reopening a view changes its generation. Validate the relevant state dependencies from V03.3, the exact view/camera/size and its visibility/interaction revisions before consuming a response; unrelated layout edits do not invalidate it. **[F] The source native path already enforces some of these guards** — native-input refuses on `profile_sha256` mismatch (“*Native profile is missing or stale; reconnect*”) and on `state_hash` mismatch (“*Native view is stale; refresh before input*”), and hidden stickers are rejected as targets via `interactive_styles()[lab] == 0`. **[P] 1.0 keeps both rules:** the integer-ID pass must share camera, size and clipping with the colour pass, and visibility must remain separate from interactivity — a ghosted object may be non-interactive while still participating fully in protection checks.

[D] Multi-layer pick disambiguation is still an open product decision (D-5). The engine can return an ordered candidate list for one ray cheaply; what the user should mean by a click — front layer, current working object, or an explicit depth choice — is not decided, and 0.4’s exact-pick contract does not answer it.

V09. Versioning and compatibility — N6

[F] Version identifiers already in the source:

Identifier	Value / source	Governs
<code>model_id</code>	<code>Model.model_id</code> , from <code>assets/manifest.json</code>	geometry, cuts, IDs, seeds, frames
<code>manifest_sha256</code>	<code>digest(canonical(model.manifest))</code>	asset content
<code>REFERENCE_VERSION</code>	<code>'exact-incidence-reference-v1'</code>	reference transport proofs
<code>NAMING_VERSION</code>	<code>'incidence-address-v1'</code>	address display and parsing
<code>FRAME_VERSION</code>	<code>adapter</code>	frame conventions
<code>macro version</code>	per record, ≥ 1	macro content
<code>workspace version</code>	<code>new_workspace(... version=1)</code>	workspace schema
<code>Session.rev / head / st.hash</code>	runtime	transaction position
<code>certificate format</code>	<code>'C600-STUDIO-CERTIFICATE-v1'</code>	preview certificates

[C] `assets/manifest.json` is an immutable model boundary; geometry, cuts, IDs, seeds and frame changes require a **new model identity and migration** (`AGENTS.md`). Migration must not silently change the model, numbering, frames or the meaning of old data.

[P] Per-data versioning and incompatibility behaviour:

Data	Binds to	On mismatch
Mechanical state / journal	<code>model_id</code>	refuse to load; never reinterpret
Protection	<code>model_id</code> + orbit numbering	refuse
Draft	<code>workspace version</code> + <code>model_id</code>	migrate additively; otherwise retain the original as an exportable, inactive draft and explain why it cannot execute
Reference certificate	<code>model_id</code> + <code>manifest_sha256</code> + <code>REFERENCE_VERSION</code>	keep the record, mark A/B recorded, require re-verify
Macro recipe	<code>model_id</code> + macro version	refuse if model differs; recipes are legal words, not names
Naming display	<code>NAMING_VERSION</code> + <code>model_id</code>	old <code>copy_text</code> stops parsing — by design , already enforced
Camera / layout / display prefs	shell schema only	reset to default silently; never blocks loading
Keybinds	shell schema + command IDs	keep, report unknown commands

[P] Rule: display-only changes never force a canonical renumber. `NAMING_VERSION` may advance for spelling or layout with `model_id` unchanged, and canonical IDs and archives are untouched. **[F] This has already happened once:** the $T^{-1} \rightarrow T^{-1}$ display change kept the internal token, shortlex order and old inputs valid.

[F] Test fixtures are model-bound artefacts. `test_mathematical_names.py` hardcodes 48927 $\rightarrow$ cell 113, 17810, 14056, 35778, 26789, (φ^3 , '1', '1', '1'), ('W', 'X', 'Y', 'Z'), 433, 600, 35, 177, 120, and orbit pairs (4, 5) (12, 13) (18, 19) (26, 27). **[F] None of these tests asserts `model_id` or `manifest_sha256` — they construct `Model()` directly.**

[P] Therefore the fixtures are correct as drift detectors but incomplete as evidence. Two changes:

1. **Bind the evidence.** Add a `model_id` + `manifest_sha256` assertion to the naming test class setup, so a passing run names the model it proves something about.
2. **Never turn a test green by editing the expected constant.** If the model genuinely changes, the procedure is: new `model_id`; new fixture file; old fixture retained against the old model; and each new constant justified by an independent derivation (incidence recomputation or a legal-witness replay), not by copying the new output.

[C]/[P] Any change to model identity, numbering or mechanical semantics is a user decision (D-3). Routine test maintenance may never be the vehicle that approves a model migration.

V10. Old/new equivalence — same full-label sample

[C] The authoritative state is an exact permutation of all 259,800 labelled slots; colour agreement, position agreement and exact-label solving are three different judgements. New and old must be compared at the label level.

[F] The pieces already exist: `GET /api/labels` returns all labels as `<u4 binary`; `session.export()` and `/api/export` produce a gzipped canonical session; `/api/certificate` emits the certificate; `state_hash` gives a full-state digest; `test_full_reference_replay.py` and `tests/reference_solve.json.gz` exist as a replay baseline. **[F] What does not yet exist is a single normalized fixture format binding state, model, frames and expected selection together** for two frontends to consume — this remains an open item (it is a prerequisite, not a release-time task).

[P] Define C600-COMPARE-FIXTURE-v1, read-only, exported from 0.4, never written back:

```
{ format, model_id, manifest_sha256, naming_version, reference_version, frame_version,
```

```

state: { head, rev, state_hash, labels_sha256, labels_blob },
workspace: { orbit, current, target, next, roles, reference, block, protected },
expected: [ { probe: "pick"|"resolve"|"effect"|"protection",
input: {...}, output: {...} } ] }

```

[P] Comparison protocol — the two programs never write the same live session. Keep the exported fixture read-only. Run static probes without mutation, and replay committed steps into separate disposable writable session copies initialized from that fixture. Each frontend receives the same command sequence; compare:

1. labels_sha256 after each committed step — exact label equality, not colour;
2. state_hash after each step;
3. resolved canonical object for each resolve probe;
4. picked canonical object for each pick probe, at the fixture's camera and viewport;
5. protection verdict and support/conflicts sets for each effect probe.

[P] Divergence in 1 or 2 halts the migration. Divergence in 4 alone is a renderer defect, not a mathematics defect, and is triaged separately — this is exactly the distinction the equivalence harness exists to make.

V11. GUI route and switching conditions

[C] Rust + eframe/egui + egui_dock + wgpu is the preferred route to validate. Qt Quick + QQuickRhiItem/QRhi is the main comparison. Avalonia stays a named alternative for a C# route and is **not** developed in parallel. Windows first; Linux/macOS later. Intel HD 620 no longer constrains anything. A dedicated D3D11 engine is not required.

[F] Existing evidence is a data-path smoke only. The offscreen wgpu probe submitted 433 slots → 10,160 triangles, an 8,660-slot 20-cell neighbourhood → 203,200 triangles, and all 259,800 slots → 6,096,000 triangles at 2560×1600, verifying manifest hashes, frame conventions, 433-region boundaries, vertex incidence and integer slot read-back ranges, on the **old HD 620 machine**, explicitly excluded from Legion performance conclusions. Not demonstrated: transparency ordering, 4-D animation, interactive picking, a real GUI, multi-window, IME, sustained frame time, device loss, or the target platforms.

[F] Hyperspeedcube is a useful reference but not a warranty. Pinned commit 63737aa467ed8eccacafaae3eec97cf1f980b794 **declares** eframe/egui 0.34.1, egui_dock 0.19.1, wgpu 29.0.1, winit 0.30.13 plus its own hcegui and hyperdraw. It shows a combination with dedicated UI/render code, not a result obtainable by adding two dependencies. **[P] Its handling of multi-window, IME and heavy text editing must be verified in its source before being counted as solved** — the previous round's caution stands.

[P] Hard gates — failing any one eliminates a candidate, regardless of other strengths:

#	Gate	Pass condition
H1	Mathematical correctness	picked/resolved canonical object matches the V10 fixture on every probe
H2	No accidental execution	during text entry/IME composition, no keystroke triggers a twist or a commit; focus loss clears held input
H3	Real independent windows	Global, Local and Keyboard are OS windows — movable to another monitor; independently minimizable, restorable
H4	Exact picking	integer-ID pass shares camera, size and clipping with the colour pass; stale read-backs discarded; hidden ≠ interactive
H5	State consistency	no path lets the shell mutate puzzle state locally; every commit goes through preview + token

#	Gate	Pass condition
H6	Clean launch	starts on a clean Windows machine with no .NET 3.5, no Managed DirectX, no MPUIlt install, no dev toolchain, no API key

[P] Soft gaps — a known cost, budgeted and scheduled, never silently deferred: font and math-glyph rendering quality; IME comfort beyond H2's correctness floor; docking polish; per-monitor DPI transitions; packaging size; dependency-update cadence.

[P] New-frontend performance is a screening target, not yet a release promise; R00 governs the current-release B4-12 deferral. 0.4's existing B4-12 thresholds (instant turn p95 ≤ 100 ms; bank/Local navigation p95 < 50 ms) remain unchanged acceptance obligations tracked in V11.1. The existing 1080p 30 FPS adaptive/full-detail rendering specification is separate; none of these measures is interchangeable. The Legion plan's 2560 $\times$ 1600 / 60 Hz / p95 ≤ 16.7 ms is **proposed, unmeasured and unapproved**. **[P] For candidate screening only,** use: default working view sustains 60 Hz p95 over three 60-second runs on the Legion with a bound NVIDIA adapter; full-detail 259,800 slots must render correctly at any frame rate; and CPU update, GPU time, present interval and interaction latency are recorded separately, never derived from a wall-clock read-back. VRAM, idle power and full multi-window budgets stay open (D-6).

[P] Switch to Qt when the preferred route fails a required gate and Qt passes the equivalent bounded sample, or when both meet the accepted screening boundary and Qt demonstrably needs less custom work across detached windows, input, math text, DPI and sustained operation. Use the same declared environment and screening criteria for both candidates. The current B4-12 deferral is not a framework pass or a hardware exception and cannot conceal a correctness or input defect. **[P]** Python Session reuse is common to both routes and is not evidence for Rust.** **[P] Toolchain friction, unfamiliarity or compile failures are toolchain evidence, not framework performance evidence.**

V11.1 B4-12 carryover: performance optimization and closure

[C] Tracking status: not passed / deferred from 0.4 into the 1.0 Architecture workstream. The user has authorized this scheduling change, not a new implementation choice or a successful performance result. Resume optimization and verification after the planned device change. Preserve the existing 0.4 failure record and the exact source/dependency/environment identity; keep new-device observations in a new record rather than replacing old results.

Acceptance contract retained

B4-12 category	Original target	Final-candidate evidence
Actual turn	p95 ≤ 100 ms	At least 100 formal observations on one frozen candidate
Bank navigation	p95 < 50 ms	At least 100 formal observations on that candidate
Local-center navigation	p95 < 50 ms	At least 100 formal observations on that candidate

[C] Preserve each category's original input/command-entry to correct, version-matched visible-surface boundary and its correctness checks. Record actual Present CPU-return or actual WM_PAINT as applicable, using the retained fixture contract. These endpoints do not establish physical-keyboard, GPU-completion or compositor-display latency. Do not move work past the completion boundary, subtract waiting already within it, substitute renderer FPS or a short function timer, or use a fast new-machine result to rewrite an old failed observation. A new frontend must explicitly demonstrate an equivalent end-to-end boundary before comparisons or closure claims.

Proposed bounded work packages

[P] The work packages below are sequential candidates, not authorization to start parallel rewrites or a multi-day investigation during 0.4 release closeout. Before each candidate, identify the hypothesis, smallest owned scope, complete invalidation dependencies, fixed screening observations, useful-benefit criterion and stop condition. Choose the next package using retained evidence and new-device screening.

ID	Scope and hypothesis	Required safety boundary and decision evidence
PERF-1	Avoid proven repeated UI updates and unchanged cycle-selector or keyboard text/layout work	Reconcile fresh selection, focus, command availability, canonical object bindings, font/DPI, scroll and accessibility against a fresh reference. Measure the changed scope and full interaction; local savings alone do not close B4-12.
PERF-2	Reduce unnecessary repeated response construction, transfer and parsing where complete dependencies permit reuse	Keep coherent model, state, workspace, guard, filter, inspection and macro versions, validated fallback and reconnect behavior. Any protocol change follows V03 and D-1; do not delete authoritative data or implement it as a 0.4 closeout shortcut.
PERF-3	Reuse deterministic derived work in the review/preview/commit path only when exact inputs and lifetime match	Keep full-label correctness, Net/Strict and position/orbit protection, fresh execution permission, durable commit/acknowledgement and recovery. Do not bypass checks or lower SQLite durability.
PERF-4	Validate presentation and all real windows on the intended device	Separate CPU, GPU, presentation and end-to-end measurements. Cover focus, real input, DPI, resizing, sustained operation and device recovery where affected. Do not infer a bottleneck or hardware remedy from device names or aggregate utilization.

New-device validation sequence

1. **[C] Establish identity first.** Record OS, CPU/GPU and actual adapter, driver, memory, power settings, resolution/DPI, visible scene, window configuration and build/source/model/dependency hashes. A device change invalidates reuse of the old environment's performance result; unchanged mathematical evidence may still be reused within its stated scope.
2. **[P] Screen the retained baseline on the new device with small fixed samples.** Keep category order, warmup policy, foreground/correct-surface checks and all attempts. This provides a baseline, not proof that hardware caused the old failure. If a causal hardware claim is needed, design an appropriate controlled comparison; it is not assumed for this workstream.
3. **[C] Validate only the candidate's affected paths plus necessary regressions.** Require exact labels/state/context, protection and permission behavior, undo/checkpoint/recovery, and applicable independent desktop checks. Reject wrong-state or stale-permission candidates regardless of speed. Preserve prior evidence and failure records; do not restart an unrelated full audit.
4. **[C] Screen before long acceptance.** Stop obviously over-budget candidates. Do not rerun unchanged failures to select the best result. Two failures with the same cause require a concrete blocker and alternative before more work on that path.
5. **[C] Freeze a viable candidate and then run the three final 100-observation campaigns.** Preserve all raw attempts, warmups, errors and the declared p95 method. Record counts, median, p95, p99 and maximum without changing thresholds. Recheck artifact/source/environment identity and applicable correctness evidence. The 2560×1600/60 Hz proposal above neither replaces nor silently adds an approved B4-12 target.

[C] Closure rule: only matching final-candidate evidence satisfying all three original targets and applicable correctness/desktop requirements closes B4-12. Until then keep its status not passed/deferred or explicitly failed for the new candidate. A newer device, merged optimization, published architecture, successful PDF build or released 0.4 package is not closure evidence. Existing undecided architecture items D-1 through D-7 remain undecided.

V12. Migration slices

[C] Migration proceeds in stages, each with a runnable deliverable and a rollback boundary. 0.4 code, model assets and personal sessions are not modified by this plan.

[P] Slice 1 — the vertical thread. Not a drawing demo. It must complete, end to end:

real Local → select the same canonical object → state an explicit frame/macro → compile the variant and show the **actual** cost → full-action and protection review → explicit commit → all views update consistently → undo/recover

Scope discipline: one orbit, one Local view plus one detached window, the existing macro library, no endgame, no cross-orbit switching, no scoring. **Rollback:** restore the pinned compatible engine/shell pair and the untouched session copy. Protocol work stays isolated and additive; deleting the shell alone is insufficient if its engine contract changed.

Slice 1 acceptance — each item is an observation, not an assertion:

1. The object selected in the real Local resolves to the same canonical ID as the V10 fixture expects.
2. copy_text round-trips: copy from the graphic, paste into a field, the same object is located and highlighted.
3. Compiled cost displayed equals emitted_primitives, and the over-limit path preserves source, frames and draft.
4. The four certificate states A/B/C/D are separately visible and never collapsed into one “certified” badge.
5. Commit goes through preview + token; a forced stale preview is refused with a stale code, not a string.
6. Undo restores the prior state_hash; a killed shell mid-commit resolves through V03.5 to a definite answer.
7. H2 and H6 hold.

[P] Slice 2 — multi-window and selection roles. Global + Local + abstract + Keyboard as real windows; the seven roles of V08; per-view cameras; async pick invalidation under resize and camera motion; layout persistence across DPI and monitor changes. Collect the N4 cache hit/miss evidence and affected PERF-1/PERF-4 observations from V11.1 here. **Rollback:** revert to slice 1 layout.

[P] Slice 3 — protection, Net/Strict and cross-orbit. Full-operation review including hidden scope and cross-orbit collateral; Net vs Strict presented distinctly before execution; cross-orbit Next with preserved drafts. **Rollback:** slice 3 is additive; slices 1–2 remain runnable.

[P] Slice 4 — endgame and worksheets. Endgame families, residual diagnosis, worksheet reuse with re-verification on state/target/version change.

[P] Slice 5 — packaging and platform. Clean-machine Windows install (H6), then real Linux/Vulkan and macOS/Metal verification. **[C]** CI compilation is not multi-window, input or GPU evidence; Vulkan-on-Windows is not Linux evidence.

[P] Gate between slices: the V10 comparison passes on that slice’s probes, and the slice runs continuously through a real solving sequence — not a button tour. Track affected V11.1 work separately at each gate; a runnable migration slice does not automatically close the deferred performance item.

V13. Decisions required from the user

These are the only items in this document that should not be implemented without an explicit answer.

#	Decision	Why it cannot be defaulted	Consequence
D-1	Transport: local HTTP + SSE (proposed) vs. framed pipe over EngineProcess	Durable protocol commitment affecting auth, cancel, reconnect and the comparison harness	Everything in V03

#	Decision	Why it cannot be defaulted	Consequence
D-2	Persist emitted_primitives/source_primitives into derived_from.proof (V04.2 Gap 1)	Changes a saved-record schema; old records would lack the field	Macro library compatibility (V09)
D-3	Any model identity / numbering / mechanical-semantics change	AGENTS.md makes this a new model identity plus migration	Fixtures, archives, all certificates
D-4	Whether user-initiated operation splitting is in 1.0 scope at all	Splitting changes Strict checking, transactions, journal and visible intermediate states (V05.2)	Endgame and large-variant workflows
D-5	Multi-layer pick disambiguation semantics (front / working object / explicit depth)	A product judgement about what a click means; determines whether picking returns one ID or a candidate list	Renderer pick pipeline design
D-6	Release performance targets: VRAM ceiling, idle power, multi-window budget; and whether 60 Hz @ 2560×1600 becomes a commitment	Currently proposed and unmeasured; must not become a promise by default	V11 screening vs. release criteria
D-7	Acceptance/sign-off split: engineering acceptance, mathematical review, UX approval, release authorization	Four different judgements with different evidence and different signers	Alpha/Beta/1.0 gating

V14. Historical source observations and evidence boundary

[F] The A1 investigation read source; it ran no application tests. A2 merges document review and the latest user priority. PDF compilation/layout checks are document checks, not application builds, native acceptance or performance evidence.

[F] Historical A1 status, retained for provenance and not a current release assessment: Final acceptance is incomplete; the full headless baseline stands at 46 groups passed / 9 failed with overall exit 1; candidate serial run 42/44 with two test-copy assertions open; and three fixes — C1 (post-commit cancellation reply), C2 (missing-frame diagnosis), C3 (mathematical-name orbit-protection entry) — remain **unmerged**. A2 does not revalidate or update those historical counts; consult the active release handoff for current evidence.

[F] Explicitly not established anywhere in this document: that any candidate GUI passes any gate; that the wgpu route meets any performance target; that transparency, animation, interactive picking, multi-window, IME, sustained frame time or device-loss recovery work; that Hyperspeedcube has solved multi-window/IME/text editing; that the naming round trip is discoverable in the current native UI; that a comparison fixture format exists; that any Linux/macOS behaviour has been observed.

[P] Terminology discipline this document follows and 1.0 should keep: *verified* (recomputed now, this model, this revision), *recorded* (provenance stored, not re-verified), *unknown* (insufficient evidence for this specific conclusion), *unchecked* (not attempted). These four are never merged, and none of them is a score.

Appendix A. N1–N7 tracking

N1

Fact established (source read)

Geometry variants are independent saved entries with `derived_from={id,version,kind,reference,proof}`; proof untrusted on import; new identity at `version=1`; `comparison=None` for geometry; `compile` already reports affected orbits; `prefix_semantics` forbids inheriting approval/cost; `preview` token bound to `(head,rev,pre_state)`, memory-only

Architecture change proposed

Name A/B/C/D with independent lifetimes (V04.2); persist emitted/source cost (**D-2**); add macro/reverify for recorded → verified; one engine-computed Availability object driving buttons, keys and API identically (V04.3)

How it gets accepted

Slice 1 acceptance #4 and #5; a stored variant displays recorded until re-verified; no path enables a control the engine disabled

N2

Fact established (source read)

MAX_PRIMITIVES is a total ceiling, WORD_LIMIT a chunk size, already asserted by test; message hardcodes 3,000,000; **no split tooling exists** anywhere

Architecture change proposed

Three labelled cost stages; exact $O(\text{distinct-moves})$ pre-pass that never allocates the emitted sequence and is cancellable on the existing protocol; over-limit preserves source/frames/draft and offers no auto-split; splitting re-framed as a user mathematical decision (**D-4**); interpolate the constant

How it gets accepted

Slice 1 acceptance #3; a forced over-limit run shows exact overrun and loses nothing

N3

Fact established (source read)

Cancel is InterruptedError; cancel_event scoped to one job under one lock with max_workers=1; _caps/_base/_maps all published only when complete; _base/_maps arrays read-only, _caps queue arrays mutable; temp primitive_words never shares a reference with a published mapping; **three existing tests already assert the cancel-safety properties**

Architecture change proposed

Written cancel-point → retained-object → next-call contract (V05.3); make _caps arrays read-only; key _caps/_base by model_id

How it gets accepted

Extend test_reference_variants.py with _base_regions cancel, post-proof/pre-publish cancel, and write-rejection assertions — **not implemented this round**

N4

Fact established (source read)

_maps LRU=4 is per engine process, not per window; windows share one service; entries are deterministic and carry no correctness role; ≈ 1.75 MB/entry plus words

Architecture change proposed

Cache contract: engine-owned, keyed by model/reference version and an ordered source/destination frame pair, single writer, capacity unchanged until measured, eviction never user-visible

How it gets accepted

Slice 2 collects hit/miss under a real multi-window cross-orbit sequence; capacity changes only on that evidence

N5

Fact established (source read)

Domain reasons are structured, but **all exceptions flatten to `{'error': str(e)} + HTTP status`**; `InterruptedError` arrives as a string; 0.4 tests themselves `assertRaisesRegex` on English words

Architecture change proposed

Typed envelope `{code, category, recoverable, retry, context, summary, diagnostic}` with ten categories (eight domain continuations plus auth/busy); `Unknown` modelled as a result property, never an error class; evidence state / score / permission modelled separately

How it gets accepted

Frontend contains no string matching; the 0.4 `assertRaisesRegex` calls migrate to asserting code

N6

Fact established (source read)

Nine version identifiers already exist; `manifest.json` is an immutable boundary; naming tests hardcode model-bound constants and `assert no model_id`; the T^{-1} change already proved display-only versioning works

Architecture change proposed

Per-data version binding table with explicit mismatch behaviour (V09); bind fixtures to `model_id+manifest_sha256`; model change $\Rightarrow$ new identity + new fixture + retained old evidence + independent derivation; never edit an expected constant to go green; model changes are **D-3**

How it gets accepted

The naming suite names the model it proves; a display-only change leaves canonical IDs and archives untouched

N7

Fact established (source read)

`parse_slot/parse_address/parse_identity` exist via a strict 4-field `copy_text`; kinds explicitly non-interchangeable; $T^{-1} \leftrightarrow T^{-1}$ normalized; short names and bare addresses are **not** parseable; failure contract is state-preserving and tested across ten cases incl. aliases and legacy integers

Architecture change proposed

Published support matrix (V07); never claim arbitrary display strings are reversible; fuzzy search, if added, is a locator producing a confirmable candidate list, never a parser; treat “API exists” and “user can find it” as two acceptance levels

How it gets accepted

Slice 1 acceptance #2 (copy from graphic $\rightarrow$ paste $\rightarrow$ locate $\rightarrow$ highlight); close the four named coverage gaps; C3 remains unmerged and is not counted

Appendix B. Related documents

Document	Role after this revision
03_ARCHITECTURE.md	0.4 contract; still authoritative where 1.0 is silent
07_NAMING_AND_RECOMMENDATION.md	Naming/scoring algorithms; unchanged
08_INTEGRATION_AND_ENDGAME.md	Integration/endgame contract; unchanged
magic600-v1-gui-study/README.md	Candidate study and audit; V11 supersedes its gate list
magic600-v1-gui-study/LEGION_SPIKE.md	Executable spike plan; V12 slices bind to it

Document	Role after this revision
docs/V1_0_OUTLOOK.md	Public outlook; summary pointer only, no detailed contracts

Appendix C. Integration record

A2, 18 September 2026. This edition merges the separate PDF review into the existing 1.0 architecture, without creating a second normative source. The latest user-confirmed release/performance priority is R00. Proposed technical clarifications remain [P]; outstanding D-1 through D-7 are not silently approved.

Review topic	Integrated location
Durable receipts, no-op ambiguity and phase-aware cancellation	V03.2 and V03.5
Schema versus mutation revision; coherent snapshots; scoped picking	V03.3 and V08
Recorded draft provenance versus executable permission	V04.3
Directional cache keys and explicit error-category scope	V05.4 and V06
Preserve incompatible drafts; separate static probes from replay	V09 and V10
Usable GUI fallback, real rollback boundaries and bounded evidence	R00, V11 and V12

The A1 source SHA256 is 48e0589dbdacaf58c4aeba2ef3f3584a43460672ad82f3e67f074b74337d4704. The original supplied download and previous reading-edition PDF are retained. This integration did not modify application code, immutable assets, personal sessions or release artifacts, and did not publish to GitHub.

A3 amendment: B4-12 scheduled into 1.0

A3, 18 September 2026. The user explicitly places the deferred B4-12 optimization into this integrated architecture for GitHub publication. R00 now records not-passed/deferred status with unchanged thresholds, replaces A2's hardware-exception and 4.5-hour proposals, and preserves the one-candidate/90-minute closeout cap without restarting it. V11.1 defines the performance backlog, next-device evidence, safety boundaries and final closure rule. V11/V12 cross-references are aligned; D-1 through D-7 are not silently approved.

This is a documentation revision. It does not modify the application, its active validation session or its release package. The A1/A2 sources and prior failures remain preserved. Public application release notes retain R00.4's single performance sentence; private diagnostic data is not part of this publication.